

reated, either via automation in VulnMapper, or by the responsible development group
1. Once the issue is linked to the DR issue, the vulnerability tracking issue may be closed

A vulnerable package which has not been fixed yet by the vendor is detected

in a GitLab FedRAMP container image

1. A container scan finding is created in GitLab
1. An issue is created to track remediation of the finding
1. The issue is automatically labelled by VulnMapper with the appropriate Vulnerability Management Labels based on fix availability data published by Red Hat
1. If the fix will likely not be made available by the vendor before SLA is due (labels indicate it is unfixed), the Deviation Request documentation should be reviewed to see if a Deviation Request should be appropriate to adjust the SLA for remediation.
1. A DR is created, either via automation in VulnMapper, or by the group responsible for remediation
1. The issue remains open, and once a fix is available, labels are updated by VulnMapper or a team member
1. The vulnerable package is updated to the fixed version and a new image is shipped
1. When the issue is remediated & deployed, the tracking issue can be closed

SECTION: security/product-security/vulnerability-management/sla-exceptions.md

SLA exception take on two different forms at GitLab, with two slightly different processes. This difference in procedure only takes effect if the asset (image, server, package) where a vulnerability finding has been detected is shipped into and used within the FedRAMP boundary or environment. Typically, at a high level, findings in FIPS compliant images will be in-scope for FedRAMP, and should be treated as such for requesting SLA exceptions.

In FedRAMP nomenclature, SLA exceptions are referred to as Deviation Requests.

SLA exceptions in general, FedRAMP or not, are vulnerability management artifacts which describe and document review of reasons a vulnerability could not be fixed within the appropriate timeframe or SLA. These timeframes are referred to as SLAs (Service Level Agreements) at GitLab and are documented in the Vulnerability Remediation SLAs handbook page. We work to adhere to these SLAs on all findings in GitLab systems and shipped assets, such as container images and packages. As part of requesting an SLA exception for a vulnerability, details about a vulnerability, it's impact, likelihood of exploitation, and any barriers to remediation will be considered and a new SLA may be assigned. All SLA exception requests must include details about the vulnerability which needs review.

If these SLAs can't be met, an SLA exception may be appropriate based on the circumstances.

Risk Acceptance & SLA Exceptions

When remediating vulnerabilities in code, container images or on running containers or servers, it is not always technologically feasible to use the latest dependency, package or container base image. Due the need to ensure the stability and performance of GitLab as a product and a

service, or due to factors outside of GitLab's control like fixes not actually being available, sometimes resolving a vulnerability will not be possible until the SLA has already been missed.

In situations such as these, raising an SLA exception request allows extending the window or "stopping the clock" on the SLA timer for a vulnerability. This allows more time to resolve the vulnerability in consideration of additional context and circumstances.

FedRAMP vs non-FedRAMP findings

This handbook page makes reference to "FedRAMP" vs "non-FedRAMP" findings. A FedRAMP finding is a finding related to an environment deployed within the FedRAMP boundary, or software components and artifacts deployed and used within the FedRAMP boundary.

For all FedRAMP findings, SLA exceptions are recorded using the Deviation Request procedure. Where deviation requests are mentioned or SLA exceptions for FedRAMP-related findings, this documentation should be reviewed for specific guidance and current reporting requirements for FedRAMP.

When is it not appropriate to request an SLA exception?

SLA exceptions should not be requested when an SLA could not be met due to reasons which were within GitLab's control. This includes vulnerabilities requiring inter-group or inter-team collaboration, team members being unavailable due to planned or unplanned absence, or other work having a higher priority (such as feature or other bug-fix work).

When SLAs can not be met in circumstances such as these, the SLA/SLO being missed and the related issues should be reviewed as part of regular group development retrospectives, similar to how bugs or features planned for a specific milestone being missed should be reviewed.

Some common scenarios or suggestions for how to prevent these kind of situations include:

- Reviewing vulnerability SLAs and time remaining as part of regular bug triage rotations to ensure SLAs are not near miss or already missed
- Vulnerability issues should be assigned and have an owner, and the owner should also ensure vulnerabilities are handed over during times of absence, planned or unplanned
- Vulnerability issues should be handled as part of regular retrospectives and milestone planning work

When is an SLA exception request appropriate?

There are a number of situations where it is appropriate to extend the SLA window or "stop the clock" on the SLA for a vulnerability.

Mitigating circumstances or controls

Sometimes vulnerabilities will be detected on an infrastructure system, software project or container image for a component which is used in such a way that the vulnerability is either not present or not exploitable. Mitigating controls in these instances either mean the impact is lower than the original severity used to calculate the SLA, or the vulnerability is not impactful at all.

Whilst it is still important we resolve this by removing any vulnerable code from code, images and packages shipped by GitLab, with mitigating circumstances we can prioritize other work over vulnerabilities in this scenario, as is appropriate for the actual impact of the vulnerability.

An example: A vulnerability in the XML-import functionality of a software library used as a dependency has an XXE vulnerability, however the project disables or does not use the XML import functionality of the library.

For non-FedRAMP findings

If updating the component can be safely delayed to make way for higher priority engineering work, then an exception issue is appropriate. Analysis explaining the mitigating circumstances or controls will be required and reviewed to make certain remediation can be safely delayed. The updated SLA should represent an updated severity rating, as this is used to calculate the new SLA. For FedRAMP findings the SLA associated with individual severities are fixed and can only be adjusted by a single severity (i.e. critical cannot be adjusted to low, only high). There is more flexibility for non-FedRAMP findings.

For FedRAMP findings

An SLA exception is not appropriate under FedRAMP guidelines for findings with mitigating circumstances or controls in place.

False positive detections

Sometimes vulnerabilities will be found in software by vulnerability scanners which are not actually present. This can be based on inaccurate detection methods, or limited context around the detection due to limited scanner functionality. False positives refer to any situation where a vulnerability should not have been detected at all, not situations where a vulnerability is present but not exploitable due to other mitigating factors. In these cases, there is no vulnerability to remediate, and the vulnerability can be treated as a false-positive detection.

For non-FedRAMP findings

An exception can be created to temporarily extend the SLA until the false-positive detection can be addressed in the scan configuration or scanner itself. If this is not feasible, then a permanent exception may also be appropriate.

For FedRAMP findings

A False Positive Deviation Request can be opened to address not meeting SLA for this finding.

Risk & severity adjustments

Software which has been repackaged by a vendor (such as operating system packages) often have a different severity rating for vulnerabilities in a package based on how the software has been packaged and the default configuration.

Mitigating controls and circumstances can also exist which impact the actual severity of a vulnerability.

In these cases, an exception to typical SLAs can be appropriate.

For non-FedRAMP findings

An exception can be created to extend the SLA and adjust the severity of the vulnerability.

For FedRAMP findings

In FedRAMP environments, there are much more specific rules for adjusting the risk of a finding, which are detailed on the Risk Adjustment section of the Deviation Request Procedure. If this handbook page indicates that it is appropriate, a Risk Adjustment DR can be created.

Vendor dependencies & fix availability

Vendor dependencies, depending on the software vendor, have their own lifecycle of reporting, analysis, and scoring which is represented in vendor advisories. For example, Red Hat releases RHSA (Red Hat Security Advisories) detailing the outcomes and fixed versions of software packaged and distributed by Red Hat. The outcome of this process can often mean that a fix will not be made available, as the vendor has found their packaged version of the software to not carry the vulnerable code, not be exploitable under default configuration, or that exploitation requires features to be enabled which are disabled in their packages.

In the situation where releasing a fix is delayed, labels or manual research of advisory information will indicate that a fix is not yet available or "unavailable". An exception to extend the SLA is appropriate in this case.

In the situation where a fix will not be released at all, as indicated by manual research or advisory information or automatically applied "will not be fixed" labels, an exception to be exempt from SLA is appropriate in this case.

For non-FedRAMP findings

An exception issue should be created for either a temporary or permanent SLA exemption, based on findings being either "fix unavailable", or "will not be fixed" respectively.

For FedRAMP findings

In FedRAMP environments, a Deviation Request should be opened. There are specific rules around how this should be handled, detailed in the Operational Requirement DR Procedure. In many cases these can be created automatically. If automated DR creation does not apply, an Operational Requirement DR can be opened manually to adjust the SLA.

Operational and technical requirements

When operational or technical requirements mean that resolving a vulnerability within SLA is either not technically feasible, or represents risk to the stability or availability of GitLab software or systems, an exception SLA may be appropriate if other mitigations can be put in place.

For non-FedRAMP findings

In non-FedRAMP environments, an exception issue should be created so Security and the responsible development group can collaborate on potential mitigations and if appropriate extend the SLA for remediating the finding.

For FedRAMP findings

In FedRAMP guidance, this is not considered a valid reason to extend or exempt the SLA of a finding. Security should work with development groups in this case to find alternative approaches which meet SLA.

SLA Exception Procedures

FedRAMP Deviation Request Procedure

For FedRAMP related vulnerabilities, all exception requests must follow the FedRAMP Vulnerability Deviation Request Procedure. For vulnerabilities in 3rd party (non-GitLab) dependencies, the SLA remediation timeframe re-starts once a patch/fix is released/published.

Non-FedRAMP Risk Acceptance & SLA Exception Procedure

If you've identified a vulnerability that is a candidate for an exception, in a non-FedRAMP image, project, or infrastructure system, please open a vulnerability exception issue using the exception template.

Please fill all out the pertinent information requested in the template. For reference, the information required is as follows:

- Vulnerability description, including CVE or other identifiers
- Priority/Severity of Vulnerability
- Original remediation due date per above SLAs
- Length of requested exception
- List of applicable assets or projects (hosts, container images, GitLab repositories, etc)

You will need to provide a high level justification of the exception and why it is appropriate. It is also helpful (and welcome!) if you can provide any technical detail you think helps explain any mitigations, compensating controls, or any other analysis which adds context around lessened impact of the vulnerability.

All of this helps us make quick decisions around how appropriate an exception is, and if there is any other way we can help to mitigate the vulnerability within SLA.

Vulnerability exceptions can be grouped into the following categories:

Exception type	Exception length	Description
----	----	----
~"risk treatment::remediate	N/A	The vulnerability will be remediated and an exception request is not required
~"risk treatment::mitigate severity::remediate"	N/A	The severity level should be downgraded due to compensating controls in place.
~"risk treatment::mitigate severity::accept"	Permanent	The severity level should be

downgraded due to compensating controls in place AND the risk should be accepted (i.e. will not be fixed). |

| ~"risk treatment::false positive" | Permanent | The vulnerability was incorrectly reported and is not actually present or exploitable in any way. |

| ~"risk treatment::operational requirement" | Temporary | The vulnerability cannot be fixed without causing operational instability, often due to a third party dependency without a patch release or dependency on a package version that would require breaking changes to upgrade. |

| ~"risk treatment::accept" | Permanent | The vulnerability should be accepted (often due to very low risk) and it will not be fixed. |

The listed labels are useful for reporting on our SLA compliance and the associated risks to GitLab, and if you are not sure which one should be applied, please ask when opening the exception request and the Vulnerability Management team will apply the correct one.

Exception length restrictions

We currently allow exception lengths based on priority/severity as follows:

P/S	30-days	60-days	90-days	365-days
~"priority::1" or ~"severity::1"	Yes	No	No	No
~"priority::2" or ~"severity::2"	Yes	Yes	Yes	No
~"priority::3" or ~"severity::3"	Yes	Yes	Yes	Yes
~"priority::4" ~"severity::4"	Yes	Yes	Yes	Yes

For exceptions which are not time based (i.e. for false positive detections) we have permanent exceptions. If you think a permanent exception might be appropriate, let us know when opening the issue and we'll review to see if a permanent exception might be appropriate. Generally speaking, time based exceptions are preferred as they give us an opportunity to explore options with you to address a finding, even if it is a false positive.

Exception approvers

After the issue is open, the requestor should assign the due date to match the appropriate SLA, based on when the linked vulnerability was first detected.

The request will be reviewed by a member of the Vulnerability Management team, and approved if appropriate. If not approved, guidance will be given on how to mitigate the risk of the vulnerability or remediate it within SLA.

Please feel free to reach out in the gsecurityvulnmgmt channel on Slack if you have any questions or need to bring any more immediate concerns to our attention.

SECTION: security/product-security/vulnerability-management/sla.md

Vulnerability Management SLAs and Labels

The following timelines or service level agreements (SLAs) are based on many factors - such as regulatory compliance, customer SLOs & SLAs, vulnerability impact, scope, prevalence in GitLab environments, impact if exploited, and defining reasonable turn-around times for mitigation and

remediation to protect GitLab and our customers. All of these factors will be considered when mapping the priority to GitLab's priority labels. All components in scope of Vulnerability Management are subject to the same SLAs. The SLAs are as follows:

CVSSv3 Score\ (TTR)	Severity	Priority	Time to mitigate	Time to remediate
-----	-----	-----	-----	-----
9.0-10.0 (Critical)	~severity::1	~priority::1	Within 24 hours	30 day SLA
7.0-8.9 (High)	~severity::2	~priority::2	N/A	30 day SLA
4.0-6.9 (Medium)	~severity::3	~priority::3	N/A	90 day SLA
0.1-3.9 (Low)	~severity::4	~priority::4	N/A	180 day SLA

\ A vulnerability's severity is assigned using the first available value from this list:

1. CVSSv3 base score.
1. CVSSv2 base score.
1. Base risk score reported by a vulnerability scanner.

A vulnerability is considered remediated when a fix has been made available to affected deployment methods. Typically this would include the fix being published to all of our SaaS environments, as well as a new self-managed release which contains the fix.

Definitions

What is an SLA (Service Level Agreement) or SLO (Service Level Objective)?

SLA and SLO are both overloaded terms, typically referring to the timeframe within which a specific outcome is guaranteed (SLA) or aimed for (SLO). These terms are used in several contexts - incident management, customer contracts, compliance frameworks, security, and GitLab's vulnerability management SLAs are also often inputs into other SLAs as a result (such as FedRAMP vulnerability remediation SLAs).

Within the context of vulnerability management at GitLab, the term SLA and SLO (Service Level Objective) can be used interchangeably. We refer to the timeframe within which vulnerabilities are targeted for remediation as Service Level Agreements (SLAs). Within the context of vulnerability management at GitLab, these SLAs are between GitLab team members and groups and track the time from detection to remediation and fix availability for vulnerability findings.

What are vulnerability remediation SLAs?

At GitLab we commit to resolving vulnerabilities in our software and our systems within specific time limits. Typically these limits are referred to as Vulnerability Remediation SLAs (Service Level Agreements) or SLOs (Service Level Objectives) depending on whether or not the time limit is contractually agreed with a third party or not, respectively. In most cases our remediation goals are objectives, however the term SLA is used to keep things simple and

consistent.

Why are vulnerability remediation SLAs important?

Remediation SLAs set reasonable expectations, based on the criticality of a vulnerability and expected impact, for how quickly GitLab users and customers can expect a security vulnerability to be mitigated. Vulnerabilities and the work required to address them require work to mitigate, and like all work, this must be prioritized. Severity and impact-based SLAs are the simplest way to prioritize this work whilst delivering the most positive impact to the security of our products, systems and in turn the security of our customers and users.

Responsible Disclosure vs SLAs

When we release and make public a fix for a security issue in GitLab, we naturally disclose the nature of the security vulnerability in GitLab. This information is useful to those who would seek to exploit these vulnerabilities. Because of this, we ensure that certain vulnerabilities are fixed on GitLab.com prior to public disclosure of GitLab vulnerabilities. In this situation, the SLA includes the time required to make the release and deploy it to GitLab.com.

More detail on the deadlines, criteria and procedure used to deploy fixes to GitLab.com is available in the security section of the release process documentation project.

For other environments, best effort should be made to adhere to this guidance by ensuring that where feasible, vulnerabilities in GitLab are patched and released prior to public disclosure of the vulnerability and the fix.

Exceptions to this policy may occur where other customer commitments and contractual obligations override the need to deploy a fix, such as agreed customer maintenance windows. For critical vulnerabilities, emergency maintenance should be pursued to ensure these vulnerabilities are applied prior to public disclosure. For other vulnerabilities, a best effort should be made to deploy fixes prior to public disclosure.

What does an SLA measure?

An SLA in the context of a security vulnerability measures the time it takes to resolve or mitigate a vulnerability. The definition of resolution and mitigation varies based on the vulnerability, where it was detected, and if it can be feasibly addressed without impacting the availability or stability of impacted GitLab assets. The requirements and definition of fixed are naturally different for vulnerabilities in code, vulnerabilities in installed software on servers, or bundled code in containers, for example. All of these types of assets & the vulnerabilities which impact them must have the following in common, if an SLA is to apply to them:

1. All vulnerabilities must be able to be fixed
1. All vulnerabilities must be able to be detected
1. GitLab must be responsible for the asset impacted by a vulnerability

For example, we cannot apply an SLA to a component or asset we are not responsible for - as we are not able to make commitments or make plans to address a vulnerability within a specific timeframe.

In the case of dependencies we bundle (in our code or images) or install (on systems we are responsible for) we are however responsible for deploying updates which address vulnerabilities, so an SLA is relevant here once an update is actually available. For vulnerabilities directly present due to configuration of systems or vulnerabilities in code which GitLab is responsible for, we are directly responsible and the SLA is applicable regardless of when a mitigation or remediation is available - as we are also responsible for defining the mitigation or remediation for the vulnerability as part of overall remediation of the vulnerability.

Where vulnerabilities are present in code or images GitLab is directly responsible for (i.e. not in dependencies) then the SLA will measure the time from initial detection of the vulnerability through to when a remediation is generally available to GitLab users and customers.

Where vulnerabilities are present in code dependencies, image components or packages on systems GitLab is responsible for, including installations of GitLab-provided images and packages on GitLab managed systems, the SLA measures the time from an updated package or image becoming available, through to when the updated image or package has been deployed to impacted systems. In the context of managed systems, the SLA measures specifically the time from initial detection through to resolution on a per-system basis, so this needs to be tracked on a per-system per-vulnerability basis to measure SLAs.

In circumstances where a vulnerability exists in code or images GitLab is responsible for (such as supported images and packages used for deploying GitLab), which are deployed to systems managed by GitLab on behalf of customers (such as GitLab Dedicated), the SLA for initial vulnerability mitigation and resolution in GitLab code, packages and images which are deployed to managed environments does not include the time taken to deploy fixed packages to managed environments. This is considered a distinct SLA which begins when a patch is made available to the teams managing those environments. This does not replace our obligations to responsibly disclose vulnerabilities in GitLab, so please ensure the security release process documentation is followed. There is more detail on this in the Responsible Disclosure vs SLAs section.

SLA Definitions

There are separate SLAs with distinct timelines:

- The SLA to release fixed packages and images addressing a vulnerability in GitLab code or images
- The SLA for taking the newly released packages
- The SLA for deploying fixed packages and images to environments managed by GitLab on the behalf of customers.

The SLA to mitigate and remediate a finding in GitLab code or images starts upon detection or report of a vulnerability in GitLab code or images, and ends when the updated packages or images are made available to the GitLab team members responsible for environments managed by GitLab on behalf of customers. The SLA for remediating the vulnerabilities in those managed environments begins upon availability of fixed packages and images, and ends when the packages or images have been deployed to the managed environment. This distinction is made to ensure GitLab can maintain our commitments to availability and stability of managed environments.

Given the typical different impact of different vulnerabilities across GitLab's code, images and systems, impact and therefore SLA may be different for the same vulnerability across different packages and components impacted by a distinct known vulnerability. SLAs need to be tracked at the component/package level in addition to per image or per system level to ensure impact can be accurately tracked and SLA accurately measured.

SECTION: security/product-security/vulnerability-management/what-does-fixed-mean.md

Overview

At GitLab, when a vulnerability finding is detected, it is detected against one or more assets (such as servers, container images or packages).

When remediating or "fixing" a vulnerability detection, it is important that a full remediation of the detected finding is made and validated as no longer being detected or exploitable prior to considering a vulnerability finding fixed.

This handbook page contains several scenarios for different types of detections, and sets out the "definition of done" and what constitutes a fixed vulnerability.

Vulnerability detections in code

When vulnerable code is detected or reported in GitLab's code, to consider the vulnerability finding fixed, we need to have addressed the vulnerability, validated it is no longer present, and made a version of the code available in packaged form to GitLab users and those self-hosting GitLab. This includes making fixes available to GitLab team members who are responsible for deploying GitLab to hosted or managed installations of GitLab. A vulnerability is not considered fixed until a version containing the fix is shipped and accessible to users. A vulnerability in code is also considered fixed before it has been deployed so long as the updated packages or images have been made available to the intended consumers of those packages. In the case of managed or hosted environments which GitLab is responsible, deployment of remediated code, via packages or images, to hosted or managed environments must be performed before we can consider the vulnerability fixed in those environments. For further clarification of how this works for SLAs, please see the SLA handbook page.

Vulnerability detections in GitLab packages or container images

Similar to vulnerability detections in code, where 3rd party dependencies we ship inside of our images or packages have vulnerabilities, they are not considered fixed until a fixed version of the package or image is available for installation by the intended consumers. A vulnerability in a package or image is still considered fixed prior to it being deployed or installed by users. In the case of managed or hosted environments which GitLab is responsible, deployment of remediated packages and images to hosted or managed environments must be performed before we can consider the vulnerability fixed in those environments. For further clarification of how this works for SLAs, please see the SLA handbook page.

Vulnerability detections in GitLab-managed infrastructure

For detections in hosted infrastructure, such as managed environments built around Kubernetes

or cloud servers, vulnerability findings are considered remediated for a given environment when there are no further detections of a vulnerability on any of the hosted Kubernetes workloads or cloud servers which make up the environment, for a specific advisory. At an individual workload level, such as a container or server, a vulnerability is considered fixed when the vulnerability is no longer detected as a finding when that container or server is re-scanned and revalidated. For detections of findings in GitLab code which is deployed to GitLab-managed environments, we treat the detection of the vulnerability in code, packages or images as distinct to the detection of the vulnerability on hosted workloads running that code, package or image from an SLA perspective. For further clarification into how this works, please see the SLA handbook page.

SECTION: security/product-security/vulnerability-management/what-is-a-vulnerability.md

What is a vulnerability?

A vulnerability describes a flaw in either code or configuration with potential or demonstrated security impact. For a flaw to be considered a vulnerability, rather than just a bug, it must have security impact, potential or demonstrated. Security impact is classified by the potential to cause software to behave in a way which would be considered unexpected by a typical user of the software or system, negatively impacting the availability or integrity of the software, or the confidentiality of information held or processed by the system. A vulnerability does not need to be demonstrated to be exploitable to be considered a vulnerability, as many vulnerabilities are exploited in very creative and unexpected ways, often in combination with other vulnerabilities to form an attack chain.

Vulnerabilities can be described in a few different ways. It is common for the detection of a vulnerability in code or in a system to be also referred to as a vulnerability. It is also common for information about a known vulnerability, and the metadata about the known vulnerability to also be referred to as a vulnerability. As an example, CVE-2022-38392 is considered a vulnerability. It describes a condition where unexpected impact to the availability of certain hardware can be caused. The impacted hard drives would be considered vulnerable, and the vulnerability could also be detected on impacted systems by a vulnerability scanner. When looking at the vulnerability report produced by the vulnerability scanner, the individual detection of CVE-2022-38392 in relation to an impacted system would also be considered a vulnerability. If this scanner was able to export a GitLab scan artifact, and the vulnerability was included, it would be represented in GitLab as a vulnerability. A vulnerability tracking issue could then be created to track mitigation and resolution of vulnerability (in this case, replacing the hard drive is the most likely mitigation). As you can see, the term "vulnerability" is often overloaded and can become confusing. To help avoid ambiguity, individual detections of a vulnerability are often referred to as "findings", "detection" or "vulnerability findings". The metadata about a vulnerability is often referred to as an "advisory" or "vulnerability data".

Advisories

Advisories are information about a vulnerability published by one or more sources, typically including the vendor or maintainers of the vulnerable software, hardware or operating system. Advisories are also published and maintained by bodies such as NVD and often have identifiers

such as CVE numbers, but other identifiers exist and are used by some advisory databases. GitLab also issues advisories for our own software and is able to issue CVEs as well. Advisory databases and data sources do not always agree on the particulars of a vulnerability, and often differ on the assessed impact or amount of information available about a vulnerability. In some cases, vendors may publish their own advisories which reference another advisory with more specific information - one example of this is RHSAs as issued by Red Hat covering the impact of vulnerabilities (typically with a CVE). RHSAs are only issued when Red Hat releases a fix, in some cases Red Hat will determine a specific vulnerability is not present in their packaged version of a component, and in this case, the vulnerability will not be fixed and the advisory from Red Hat will reflect this, but the advisory as recorded by NVD may not have this level of detail.

Vulnerability Findings

An individual detection of a vulnerability by a security scanning tool (such as a container scanner, or dependency scanner) is a finding. An individual vulnerability finding on a system or in a codebase can be remediated. A vulnerability finding also typically is linked to and refers to an advisory which gives guidance and impact information about the vulnerability useful when performing triage and resolving an individual vulnerability finding. At GitLab, we track vulnerability findings in GitLab as vulnerabilities, however we also use issues currently to track the workflow and lifecycle of vulnerability remediation in code and systems we are responsible for. This is different to an advisory, as the finding refers to a single instance of a detected vulnerability in a detected component, rather than general information and advice about a software flaw which may or may not impact individual systems or code projects.

SECTION: security/product-security/vulnerability-management/why-should-we-fix-vulnerabilities.md

Overview

Sometimes we encounter vulnerability findings in GitLab or GitLab systems, and through various mitigating factors or a lack of clear vectors for exploitation, it may not seem important to resolve those vulnerabilities, when viewed in isolation. The handbook page details potential reasons team members might not have considered for resolving vulnerability detections in a number of scenarios.

There are several important reasons for fixing vulnerabilities:

1. To remediate direct threat of vulnerabilities being exploited in GitLab or GitLab systems
1. To improve the security of our software and systems, to prevent negative impact to the availability and confidentiality of GitLab user data, using exploits or combinations of exploits we may not yet be aware are possible
1. To demonstrate our commitment to a mature and secure software delivery lifecycle (SDLC). This helps to demonstrate to GitLab users that they can trust our software and systems, by ensuring scans of our systems, images, and packages are as clean as possible as a measure of the health of our procedures

Mitigating factors or a lack of direct exploitation scenarios should not mean we don't need to

fix a vulnerability detection. If you believe a finding to be mitigated or not exploitable, this can be used to justify an exception to regular SLAs, however this should still be addressed and a fix shipped per the above reasons. If you believe a detection is made in error, which is typically referred to as a false positive in a security scan, you can raise an SLA exemption documenting to error in detection. Ideally, we would then feed information about the detection back to development groups (for GitLab security scanning product features) and to third party vendors or projects for all other scanners, to get the false positive resolved at the source.

SECTION: security/product-security/vulnerability-management/runbooks/_index.md

This handbook page lists runbooks created by the Vulnerability Management Team

Runbooks

- Fixing Vulnerabilities
- So you have a vulnerability finding, what next?
- Weekly Triage

SECTION: security/product-security/vulnerability-management/runbooks/fixing-vulnerabilities.md

Overview

The purpose of this runbook is to define common sources and types of vulnerability, and typically the work required to remediate the vulnerability finding.

This runbook is divided into sections based on the source of a vulnerability findings, and further broken down into types of sources. For example, it is common for vulnerability findings to originate from automated scans in CI pipelines. There are multiple subtypes of automated security scans, such as container scanning, dependency scanning, secrets scanning which all have their own sub-sections detailing a typical resolution workflow.

Source: Security Scanners

Automated security scanners are the source of the majority of vulnerability findings at GitLab. These can range from infrastructure security scanning, or scans run against GitLab assets we ship out to users, such as container images and software packages we publish.

Source: Container Scanners

Container scanners will scan built container images for software components, binaries, and OS package metadata (such as RPM or .deb package databases) embedded in the image for known-vulnerable components. Some scanners will also detect language libraries such as Ruby gems, Go libraries and Python modules which carry known vulnerabilities and have been deployed inside an image. In general, for container scan findings, if a fix has been released, updating the detected component to the version indicated as fixed alongside the detection will address the

finding, and no further work will be required. Complications can arise if upgrading a specific component causes incompatibility with other components and deployed code inside the container, such as incompatible Ruby gem versions not yet being tested against GitLab in a GitLab-published gitlab-ee or gitlab-ce image. In these cases, further work may be required to validate the upgrade will not cause any problems if upgraded.

Type: 3rd Party OS Dependency

In the case of a vulnerability finding naming OS packages present inside a container image as vulnerable to a given vulnerability (i.e. a CVE), one of these two remediation options will often apply:

- Updating any base images sources from vendors, or base images which in turn are based on vendor images, to the latest vendor-released base.
- Using the package manager (i.e. apt, yum or microdnf) to upgrade to the fixed package version as part of either a base image build or the build of the impact image

Normally, using the package manager to upgrade packages, if available for the type of base image, during build time, is considered good practice, as it will often prevent findings like this from being detected at all if the image is built frequently enough and the vendor is pushing fixed packages quickly enough. Typically updated packages will be made available before updated base images from most vendors. If they are not made available in a timely manner, an SLA exception may be appropriate. In case there is no package management, as is often the case in distroless images, the only remediation may be to update the base image from the distroless image vendor, or build a fixed version ourselves as part of the image build process. This last option should only be pursued, given the time investment, if a fix is needed and the vendor has indicated they will not be making a fix available.

Type: Language libraries/dependencies

Some container scanning tools have the ability to detect vulnerabilities in language libraries (such as Python modules, Ruby gems) or even to extract package information out of compiled Go binaries. These scanners will raise vulnerability findings for vulnerable components if they are detected embedded into a container image, as is often the case when built software such as GitLab is deployed as a ready-built container image for distribution to users.

Similar to 3rd party OS dependencies, the remediation here is typically to update the impacted library or module, and ship an updated container image to users. If there are delays in a vulnerability being addressed in a component we depend on, it may be necessary to open an SLA exception or assess alternate libraries which are not vulnerable, but provide similar functionality. Balancing the effort to move to different library should be done against the likelihood the vulnerability will be addressed upstream in the existing library, and how long that is likely to take.

Source: Dependency Scanners

Dependency scanners, unlike Container Scanners typically operate directly on project source code, prior to container images or packages being built. Dependency scanners will detect vulnerabilities in language dependencies (such as Python modules, Ruby gems, etc.) and typically, but not always, recommend steps to remediate the vulnerability. In most cases, this

will involve using the language package manager to upgrade to a newer version of the dependency including the fix, and this is all that is required to resolve the vulnerability finding. In cases where dependency upgrades break builds for projects or introduce stability or functionality problems, additional work may be required to upgrade to the newer dependency. In these cases, it may be appropriate to see if alternate libraries can be used, or opening an SLA exception explaining the interoperability issue with the upgraded dependency. In some cases, these findings may refer to dependencies which are no longer required or can be removed with small changes to the project, in this case, the most appropriate action may be to remove the dependency entirely, which will also resolve the vulnerability finding.

Source: SAST

SAST findings typically represent known and suspected code patterns which may introduce a vulnerability into a codebase. Typically, these findings will have remediation advice associated with them based on the specific finding, however impacts to the functionality of the code where the vulnerability is detected should be assessed by the appropriate development group to ensure the advice given by the scanner will not cause any other issues if applied. Often, there are multiple ways to address SAST findings, each with different complexity, benefits, and negatives which should be considered.

Additionally to this, if development groups have novel ways in which a SAST vulnerability finding can be effectively mitigated, it may be necessary to add specific code comments instructing the scanner to ignore a specific code pattern in the future, even once a solution is applied. Many SAST scanners look for simple patterns in code or the usage of specific language libraries, and even code which is secure can sometimes generate findings.

Further to this, if the vulnerability finding is not actually exploitable to begin with, and the code highlighted in the finding is not vulnerable, adding the code comments/annotations to the detected code to instruct the scanner to ignore this code for this finding in the future may be necessary. If the finding is not exploitable (a false positive) and the code comment/annotation results in the finding not being detected upon rescan, that is also an acceptable solution.

Source: Secret Detection

Secret scanners detect data in projects which potentially represent sensitive information such as credentials, certificates or other sensitive information which should not be stored directly in a project's repository. In most cases, the appropriate action is to remove the secret, rotate any certificates or credentials which had been detected, and use proper secret storage to store this information. When removing secrets from a project, it is also important to ensure the project's version control history is also scrubbed to remove any trace of the secret. Tools such as the BFG Repo Cleaner can be incredibly useful in automating this process.

Source: IaC Scanning

IaC tools such as Terraform allow the definition and deployment of infrastructure by representing the infrastructure as code. IaC security scanners detect common infrastructure misconfigurations in the code prior to it being deployed to real infrastructure. Typically these findings will also carry suggestions on how to adjust the code to remove the infrastructure vulnerability which would be introduced if the IaC was applied/deployed.

Typically these recommendations must be reviewed in the context of the infrastructure being described, and simply applying the recommendations should be reviewed to ensure there will be no impact to infrastructure by the suggested changes. There may be other ways detected infrastructure vulnerabilities can be addressed based on the impact described in the finding. It's also possible for false-positives to be detected, as sometimes these findings do not consider the broader context of a configuration and the potential impact of the configuration is mitigated by some other part of the configured infrastructure, in this case, an SLA exception may be appropriate.

Source: Infrastructure & Network Vulnerability Scanners

Infrastructure and network vulnerability scanners scan already deployed infrastructure, from either an internal or external attack surface perspective, and report any potential vulnerabilities detected in systems or hosted application code. Some of these scanners will have access to deployed hosts either via SSH keys, or via disk snapshots, so can provide a level of inspection much higher than typical external scanning tools. Many different types of findings can be detected by these scanners, depending on their scope and configuration. This section has been split into finding type to better explain the different use cases and how to fix these type of findings.

Type: 3rd Party OS Dependency

Similar to the advice for Container Scanners, many network and infrastructure scanners can directly inspect files on the filesystems of deployed workloads (servers, serverless, containers), container images and VM images. In this case, they will either fingerprint OS packages by accessing OS package metadata in the filesystem, or fingerprinting binaries directly. The way to remediate these types of findings typically involves one of two things:

- Updating (patching) the OS of running workloads to ensure the latest OS packages are installed, and images updated (VM or container)
- Removing unneeded packages from the workload or images

Normally, using the package manager to upgrade packages, if available for the type of base image, during build time, is considered good practice, as it will often prevent findings like this from being detected at all if the image is built frequently enough and the vendor is pushing fixed packages quickly enough. Typically updated packages will be made available before updated base images from most vendors. If they are not made available in a timely manner, an SLA exception may be appropriate. In case there is no package management, as is often the case in distroless images, the only remediation may be to update the base image from the distroless image vendor, or build a fixed version ourselves as part of the image build process. This last option should only be pursued, given the time investment, if a fix is needed and the vendor has indicated they will not be making a fix available.

Type: Language libraries/dependencies

Network & infrastructure scanning tools which have filesystem access for workloads typically are also able to find and fingerprint language-specific library dependencies and any vulnerabilities impacting them. These out of date & vulnerable libraries can come from multiple sources:

- Libraries installed to support deployed applications on the server. Typically to resolve these findings, the application itself needs to have the finding addressed, and then the updated version deployed to the workload.
- Libraries installed as part of tools used to administer the workload. This can include configuration management tools, backup tooling, or diagnostic tools. In this case, the tools should either be removed or upgraded to a non-vulnerable version.
- Backups of applications or tooling made during routine maintenance. These are easy to forget about once a change or incident is over, but can be picked up by vulnerability scanners and should be cleaned up if no longer needed.

Type: Secret detection

Network & infrastructure scanning tools are often able to detect two types of secrets:

- Secrets stored on the filesystem of deployed workloads. These may be necessary for the function of applications, or they may not be. In the case of necessary secrets being detected or genuine false positive detections, an SLA exception may be appropriate. Otherwise, the secrets should be removed and stored in a supported secret storage and management tool.
- Secrets exposed by deployed applications. Typically, if an application endpoint is exposing a secret this is unintended, and the application will need to be fixed and then redeployed so that the secret is no longer exposed. If the application is a 3rd party tool or the issue can not be addressed quickly, limiting network access to the endpoint may also be an appropriate workaround or mitigation.

SECTION: security/product-security/vulnerability-management/runbooks/so-you-have-a-vulnerability-finding.md

Overview

The purpose of this runbook is to define the distinct stages of addressing a vulnerability finding.

Overall, this process is broken down into several major phases, as shown as numbered stages in the below flow diagram.

Each numbered phase is further described in a section of this runbook.

Not all phases will be applicable to every finding, as the work required to fully remediate a vulnerability finding will differ based on the actual finding.

The environment where the vulnerability is detected, the complexity of remediation, and the type of asset (container, package, image, etc.) where the finding is detected, can all influence the steps required.

Remediation Flow

mermaid

flowchart TD

A[146; Analysis] -->|Remediation is complex| B[246; Mitigation]

A -->|Remediation is simple| C[346; Remediation]

B --> C

C --> D[446; Build & Validation]

D -->|Server, Container| F[646; Deploy]
D -->|Package, Image| E[546; Publish]
E --> F

Vulnerability Remediation Phases / Steps

1. Analysis

During this phase of vulnerability remediation the impact of the vulnerability is assessed in the context of the code, package, image or infrastructure where it has been detected. Depending on who is alerted to the vulnerability detection first, this may be initially performed by GitLab Security, or other GitLab team members.

The goal of this step is to understand:

- What is required to exploit a vulnerability
- If exploitation is possible, and its likelihood
- The impact of successful exploitation
- The general complexity of the changes required to prevent exploitation

If the vulnerability finding is analyzed and found to be not exploitable, this does not remove the need to address the finding. It lowers the impact of the finding, and in turn can be used to adjust the SLA for resolving a vulnerability finding. For more detail on this, please see the [Why Should We Fix Vulnerabilities?](#) handbook page.

If assistance is required in assessing impact, please reach out to Product Security.

Once the impact is understood, a high level plan to fully remediate the finding should be documented on the associated GitLab issue used to track the vulnerability finding.

2. Mitigation

This step is typically not required for most vulnerability findings. Typically, you'll focus on remediation of the finding.

A mitigation is any change to the system, container, built image, package or code which reduces or removes the risk of exploitation for a vulnerability finding. This differs from a remediation in that it doesn't remove the vulnerability. If you think about a building being renovated to add more secure door locks, a mitigation would be building a big wall around the building whilst the remediation work (upgrading the locks) happens. Mitigations should only make sense if it makes sense to add them to address risk, as it may make more sense to instead put that effort directly into remediation work if the risk is already low, or the remediation is simple and can be performed quickly.

If a vulnerability finding is complex to remediate or there are other logistical challenges to fully remediating a vulnerability finding, it may be appropriate to add mitigations which prevent exploitation or reduce the risk or impact of exploitation to an acceptable level. If the vulnerability finding cannot be remediated within the appropriate SLA, this can be a good point to decide if a mitigation should be put in place.

3. Remediation

Remediation refers to the work required to fully remove the cause of a vulnerability finding from the system, container, image, package or code where it was detected. The actual work required will differ based on the vulnerability finding and where it was detected.

For many findings coming from external or internal research, code changes will be required to remediate a vulnerability.

For advice on remediating findings from specific scanner types, and for specific finding types, see the [Fixing vulnerabilities runbook](#).

Remediation should always be performed, even when a mitigation is in place, as it reduces the risk of vulnerable code paths or dependencies remaining resident in code, images or systems where they may become a useful part of an exploit chain (Classification) in the future.

For more specific information based on finding and scanner type, see the [Fixing Vulnerabilities runbook](#).

4. Build & Validation

Depending on the impacted asset (container image, running container, server, package or code), and the finding, the steps to validate the fix has been applied correctly will differ. The purpose of validation is to work out how to validate the change is effective, and the vulnerability finding is no longer detected. Typically, this will require rebuild of impacted images and packages so the updated images or packages can be tested.

In the case of container image scanning, validation can be as simple as re-running the scanner manually on the image, or rerunning the pipeline which generated the finding originally, after updating packages in the image or the base image itself.

In the case of dependency scanning, validation can involve pushing updated branches or running dependency scanning tools directly to validate the finding is no longer detected, after updating impacted libraries.

For deployed servers and containers, it may be required to test the upgrade first on a test server or a test Kubernetes environment to ensure impacted components are no longer detected. If findings have come from infrastructure scanning tools, such as Wiz, it may be required to review the test environment where the changes are being validated to ensure the finding is no longer detected. It is recommended that any changes to production systems are tested in a testing environment as a form of validation prior to deploying changes or updates.

5. Publishing and 6. Deploying Fixes

For security issues being remediated and made available in GitLab code or images, please refer also to the [GitLab Security Release Project](#) for additional documentation and guidance on how to follow the GitLab release process.

Once the remediation has been validated, based on the type of asset (server, container, container image, package or code) impacted, it is typically required that for built artifacts (container images, packages), a new version is published with a fix. This is like any other

software bug in this way, and regular build and publishing steps should be followed to make the fixed asset available to users of the image or package. The published container image or package may also need to be published if the finding was detected on a deployed server or container, as publishing a built asset is usually a prerequisite step to deploying the changes to the impacted server or running container.

In the case of published assets like container images and packages, typically publishing is where this process ends, as a new release is made and users can upgrade to access fixes.

In the case of deployed systems and environments, deployment tasks will be required, including change control as appropriate, to deploy the updated code, packages, or images to the impacted servers or containers. In the case of vulnerabilities in GitLab-owned code, images and packages, we will typically be dependant on those assets being published before deployment can happen. In the case of OS & container base image vulnerability findings in 3rd party dependencies, it may be possible to skip the publishing step, as no new images or packages need to be published, and instead use built-in update mechanisms (such as OS package managers) to update the vulnerable dependencies directly.

Once these steps are concluded as appropriate, the findings should no longer be detected by the scanner or internal or external reporter who reported them, and once that is validated, the vulnerability finding can be considered fully remediated, and closed.

SECTION: security/product-security/vulnerability-management/runbooks/weekly-triage.md

Overview

The Vulnerability Management team receives, reviews, and approves/denies various requests including for Deviation Requests or SLA Exceptions. To ensure timely consideration of these requests, a team member is assigned randomly on a rotation as the DRI for the week for the initial triage and handling of new requests for that week.

Automation creates a new Weekly Triage Issue at the start of each week, and randomly chooses a team member to assign it to. The issue is based on this issue template which includes

General Guidance

TODO

Specific Guidance

POA&M

TODO

SLA Exceptions

TODO

SLA Exception near breach or breached

As part of the weekly triage rotation, a review of approved SLA Exception's nearing their due date is performed. These steps are here to guide the process and provide consistent templated messaging to the SLA Exception request owner.

For an exception to fall under the At risk SLA Exceptions table, the due date must fall in the 2 weeks prior to the Weekly Triage Rotation issue due date and be in an open state.

1. Add a comment to the SLA exception using the template below.

PLAINTEXT

Hi @<author>,

This SLA Exception is at risk of breaching due date.

Has remediation work been completed?

Can this exception request be closed?

If not, please provide

- An expected remediation timeline.
- Detail why the existing extension was missed.
- A requested to update the exception length if need be.

note: An extension to the initial exception length is not a given and will require a approval by the Vulnerability Management Team.

If all remediation work has been completed and deployed

- Please confirm this is the case.
- Please close close the SLA Exception Request issue.

Thank you for your time.

The Vulnerability Management team

Deviation Requests

TODO

Wiz issue/finding

TODO

SECTION: security/product-security/vulnerability-management/vulnerability-management-team/_index.md

This handbook page describes how the Vulnerability Management team works on a day to day, quarter to quarter basis.

We recognize this process is iterative and requires regular introspection to ensure we are always improving and not stagnating in our approach.

Mission

The Vulnerability Management team enables GitLab to ship secure software to customers by providing comprehensive visibility into security vulnerabilities and driving efficient remediation through automated workflows, standardized processes, and clear risk communication. Our team serves as the central hub for vulnerability intelligence, tooling, and metrics that empower teams to proactively manage security risk across GitLab's evolving technology landscape.

Value Proposition

We provide automated vulnerability detection, standardized remediation workflows, and comprehensive risk visibility so that stakeholders and customers can confidently build and deploy GitLab securely while maintaining development velocity and meeting compliance requirements.

Scopes and Responsibilities

Primary Areas of Ownership

Vulnerability Management Standards & Procedures

- Establishing and maintaining vulnerability management policies
- Defining SLA frameworks and remediation timelines
- Creating exception and risk acceptance processes
- Setting scanning requirements and coverage standards

Vulnerability Detection & Correlation

- VulnMapper development and maintenance
- Integration with multiple vulnerability data sources
- Normalization of vulnerability findings
- Advisory data management and correlation

Automated Workflow Management

- Creation and routing of vulnerability tracking issues
- Exception request handling

Program Coverage & Visibility

- Infrastructure vulnerability scanning (GitLab.com, Dedicated)
- Container and dependency scanning oversight

Program Metrics & Reporting

- Stakeholder-scope risk communication (See Metrics below)

Customer-facing artifact generation
Supporting compliance audit evidence collection

Supporting Efficient Remediation

Identifying and documenting challenges to remediation
Automating remediation workflows
Providing documentation

FedRAMP

Vulnerability scanning tooling
Automated generation of vulnerability artifacts

Shared Responsibilities

Vulnerability Triage

The vulnerability triage model is distributed across teams based on expertise and domain knowledge. This approach improves efficiency and accuracy of vulnerability assessment.

Vulnerability Management

Providing automated contextual enrichment for all vulnerabilities
Supplying patch availability information and vendor triage status
Maintaining integration with advisory data sources
Ensuring consistent labeling and workflow routing

Application Security

Triaging HackerOne reports and bug bounty submissions
Assessing exploitability and impact for application vulnerabilities
Validating SAST and DAST findings

Infrastructure Security

Triaging cloud/infrastructure misconfigurations
Validating Wiz cloud security alerts
Prioritizing infrastructure remediations

Engineering

Triaging GitLab tooling findings related to their application components
Providing business context for vulnerability risk assessment
Assessing technical feasibility of remediation approaches
Validating dependency vulnerabilities in their codebase

Engagement Model

Triage SLAs defined by vulnerability severity
Cross-team collaboration coordinated through GitLab issues
Automated triage workflows through VulnMapper

Out of Scope

Direct Vulnerability Remediation

Tasks owned by AppSec/Engineering/Infrastructure:

- Writing code fixes for vulnerabilities
- Deploying patches
- Making infrastructure changes
- Direct system modifications

Vulnerabilities related to user end points

Owned by CorpSec

- End user system asset management
- Patch tracking and measurement on end user systems
- Reporting of end user system vulnerabilities

GitLab platform Vulnerability Management features

- GitLab Security Dashboard / Report features used by customers
- Development/Maintenance of CI/CD vulnerability scanning tools
- The GitLab advisory database
- Owned by various teams in the Sec section depending on feature.

Contacting Us

Slack

- gsecurityvulnmgmt - public team channel for questions and follow team communication
- threatvulnmanagement - private team channel increasingly used primarily for team automation
- @vulnerability-management - Slack group handle

GitLab

@gitlab-com/gl-security/product-security/vulnerability-management

FY26 Strategic Initiatives

FedRAMP

- Maturing program focus areas
- Program Advocacy

Planning

Vulnerability Management follows GitLab product milestones

Prior to a new milestone beginning, planning is performed to define the expected outcome of the next milestone, and the work required to accomplish it. We review our ability to deliver this work within the milestone and commit/adjust to issues to fit. Additionally, any work which is not scoped or not sized is qualified and labelled.

How do we breakdown quarterly items into appropriately sized work items (issues)?

Ideally, sizing and planning labels are added as new issues are created. We aim to define work as small actionable issues to assist with fitting work into milestones and being able to parallelize work. Where work has not been sized and scoped, we do this sizing and scoping work during milestone planning for items we want to schedule for that milestone.

Refinement & Sizing

Work is sized based on the estimated percentage of a milestone required to complete the task as it is defined. Work is defined as clearly as possible to try and minimize the impact of misunderstood requirements leading to scope creep. Issue sizing is performed on a defined scale of issue weights and sizing labels.

These steps are used as guidelines:

1. Check the issue for sufficiently detailed requirements needed to accomplish the desired outcome.
2. If the requirement